CortexAI Frontend Application Documentation

This document explains the setup, state management, components, features, and the detailed file-to-file execution workflow of the React client (Vite SPA) in CortexAI.

---

1. Frontend Architecture: A Beginner's Guide

The frontend is a Single Page Application (SPA) built using **React**, **Vite**, and **Tailwind CSS**. Its architecture relies on three core pillars:

1. **API Client & Identity Providers**: Handles connections to Google (via Firebase Auth) and the backend Gateway (via Axios).
2. **Global State (Redux Toolkit)**: Maintains the user's login profile and list of conversations in a single global memory store so any component can access them immediately.
3. **UI Layout Components**: Renders the login overlay, chat dashboard sidebar, active message area, and code artifact cards.

---

2. Variables & Functions Dictionary

Connection Utilities

- `auth` & `googleProvider` (instances in `utils/firebase.js`):
  - **What they do**: Configures the client-side Google OAuth popup handler.
- `api` (axios instance in `utils/axios.js`):
  - **What it does**: Custom HTTP request client pointing to the Gateway on Port 8000.
  - **Settings**: Configures `withCredentials: true` so the browser automatically handles the `session` cookie.

Global State Slices (Redux)

- `userData` (variable in `userslice.js`):
  - **What it stores**: The logged-in user profile JSON object.
- `conversations` (array variable in `conversationslice.js`):
  - **What it stores**: Holds the list of the user's past chat history sessions.
- `selectedConversation` (variable in `conversationslice.js`):
  - **What it stores**: Tracks the conversation log currently active in the chat panel.
- `setUserdata`, `setConversations`, `addConversation`, `setSelectedConversation` (Redux actions):
  - **What they do**: Dispatcher functions used by React components to modify the state variables listed above.

React Core Files & Features

- `App.jsx`: Initiates session recovery checks on page mount using the `useEffect` hook.
- `Home` (component in `home.jsx`): Checks `userData` Redux state. Renders a Google Login button overlay if `null`, or mounts the workspace layout if populated.
- `googlelogin` (function in `home.jsx`): Triggers popup authentication and extracts Google's JWT token payload.
- `handlelogin` (function in `home.jsx`): Submits the token to the backend `/auth/login` and updates Redux state.
- `Sidebar` (component in `sidebar.jsx`): Lists active conversation items, highlights selected items, displays the user's profile card, and handles "New Chat" and "Logout" clicks.
- `getcurrentuser`, `getConversations`, `createConversation`, `logout` (feature modules): Call Gateway endpoints to restore profile, read histories, open new chats, or terminate sessions.

---

3. Feature API Modules & Data Functions (Detailed Spec)

These async features act as the bridge between React UI components and the Gateway API:

- **`getcurrentuser`** (in `features/getcurrentuser.js`):
  - *Imported in*: `App.jsx`.
  - *What it does*: Sends a GET request to `/me`.
  - *Returns*: A Promise resolving to the user profile JSON object (containing `_id`, `name`, `email`, `avatar`) if successful, or `null` if the session is expired or invalid.
- **`getConversations`** (in `features/getconversation.js`):
  - *Imported in*: `sidebar.jsx`.
  - *What it does*: Sends a GET request to `/chat/get-conversation`.
  - *Returns*: A Promise resolving to an array of conversation objects.
- **`createConversation`** (in `features/createconversation.js`):
  - *Imported in*: `sidebar.jsx`.
  - *What it does*: Sends a GET request to `/chat/create-conversation`.
  - *Returns*: A Promise resolving to the newly created conversation object (`_id`, `title`, `userId`).
- **`logout`** (in `features/logout.js`):
  - *Imported in*: `sidebar.jsx`.
  - *What it does*: Sends a request to `/auth/logout`.
  - *Returns*: A Promise that resolves once the backend deletes the Redis session key and wipes browser cookies.

---

4. Redux Slice State Schemas & Mutations

The local store is configured using Redux Toolkit slices. Below is the blueprint of how actions mutate state:

A. User Slice (`userslice.js`)

- **Slice Name**: `"user"`
- **Initial State**:

```
{ userData: null }
```

- **Reducer Actions**:
  - `setUserdata(state, action)` :

        - *Behavior*: Mutates `state.userData = action.payload`.
        - *Payload*: Takes a User object `{ _id, firebaseUid, name, email, avatar }` or `null`.

B. Conversation Slice (`conversationslice.js`)

- **Slice Name**: `"conversation"`
- **Initial State**:

```
{ conversations: [], selectedConversation: null }
```

- **Reducer Actions**:
    - `setConversations(state, action)`:
        - *Behavior*: Mutates `state.conversations = action.payload`.
        - *Payload*: Takes an array of Conversation objects `[{ _id, title, userId, createdAt }]`.
    - `addConversation(state, action)`:
        - *Behavior*: Mutates `state.conversations.unshift(action.payload)` (inserts the new item at the top of the array).
        - *Payload*: Takes a single Conversation object `{ _id, title, userId }`.
    - `setSelectedConversation(state, action)`:
        - *Behavior*: Mutates `state.selectedConversation = action.payload`.
        - *Payload*: Takes a Conversation object `{ _id, title, userId }` or `null`.

---

5. State-Driven Conditional Layout Routing

Instead of relying on URL routes (like `react-router-dom`), CortexAI utilizes a pure state-driven conditional routing pattern:

- **Logic Root (`App.jsx`)**:
    - *Condition*: Evaluates `userData` state from the Redux store.
    - *Behavior*:
        - If `userData` is `null` (or loading), React renders the login overlay (`Home` component in `home.jsx`).

        - If `userData` contains the logged-in profile, React unmounts the login screen and renders the main application workspace (`Sidebar` , `ChatArea` , and `Artifact`).
- **Benefit**: Ensures that users cannot bypass the login screen by changing URL endpoints in the browser bar, while maintaining a fluid single-page dashboard experience.

---

6. Global Integration (Where, When, & How Variables and Actions are Imported & Used)

The Redux states and feature APIs are imported and shared across multiple UI components to coordinate rendering:

A. Core Redux Setup Setup

- `Provider` **(React-Redux Wrapper Component)**:
    - *Where*: Imported and run in `main.jsx` from `react-redux` .
    - *Usage*: Wraps around `<App />` and takes the compiled `store` as a prop.
    - *Why*: Generates React context parameters under the hood so that descendant child components can read state and dispatch actions.
- `store` **(Redux configuration)**:
    - *Where*: Configured in `redux/store.js` and imported in `main.jsx` .
    - *Usage*: Combines separate slice reducers from `userslice.js` and `conversationslice.js` .

B. Redux Action Imports & Usage

- `setUserdata` **(User slice action)**:
    - *Where*: Imported in `App.jsx` and `home.jsx` from `./redux/userslice.js` .
    - *Usage*:
        - Tied to the `useEffect` hook in `App.jsx` to load profile data after auto-login runs.
        - Tied to the login handler in `home.jsx` to set profile data after successful Google validation.

        - Tied to the logout handler in `sidebar.jsx` to reset the user profile state back to `null`.
- **`setConversations` (Conversation slice action)**:
    - *Where*: Imported in `sidebar.jsx` from `../redux/conversationslice.js`.
    - *Usage*: Called inside a `useEffect` hook after fetching past chats from the backend Chat Service, populating the local array list.
- **`addConversation` (Conversation slice action)**:
    - *Where*: Imported in `sidebar.jsx` from `../redux/conversationslice.js`.
    - *Usage*: Fired when the user clicks the "New Chat" button to instantly prepend a new conversation tab into the sidebar.
- **`setSelectedConversation` (Conversation slice action)**:
    - *Where*: Imported in `sidebar.jsx` from `../redux/conversationslice.js`.
    - *Usage*: Fired when clicking any conversation bubble in the sidebar to change the active chat display context.

C. Global State Selector Consumption (`useSelector`)

- **`userData`**:
    - *Where*: Read in `home.jsx` and `sidebar.jsx` via `useSelector((state) => state.user).userData`.
    - *Usage*: Hides the login panel overlay in `home.jsx` when present, and populates the avatar, name, and email details at the bottom of `sidebar.jsx`.
- **`conversations` & `selectedConversation`**:
    - *Where*: Read in `sidebar.jsx` via `useSelector((state) => state.conversation)`.
    - *Usage*: Loops through `conversations.map()` to display chat items and compares `selectedConversation?._id` to apply high-contrast focus styling on the chosen item.

---

7. CSS Styling System (Tailwind Utility Application)

The UI matches modern design aesthetics using Tailwind utility classes injected directly inside component JSX files:

- **Login Overlay (`home.jsx`)**: Uses classes like `flex`, `items-center`, `justify-center`,

`min-h-screen`, `bg-gray-950` to build a dark page container, and styling classes like `bg-white/10`, `backdrop-blur-md`, `border`, `border-white/10` to form a glassmorphic central login card.
- **Dashboard Sidebar (`sidebar.jsx`)**: Combines sizing controls (`w-64`, `h-screen`, `flex-col`) with layout positioning (`fixed`, `left-0`, `top-0`) and style tokens (`bg-gray-900`, `border-r`, `border-gray-800`, `text-gray-200`) to construct the persistent navigation drawer.
- **Interactive Controls**: Buttons and chat navigation items consume transition utilities (`transition-all`, `duration-200`, `ease-in-out`, `hover:bg-gray-800`, `cursor-pointer`) to enable smooth hover micro-animations.

---

8. Frontend-to-Backend Interface (How They Communicate)

Below is the technical detail of how React handles communication with the backend services via the Gateway:

A. Authentication & Onboarding

1. **Google Popup Login**:
   - *Frontend Function*: `googlelogin()` in `home.jsx`.
   - *Backend Service*: Auth Service (Port 8001) proxied through the Gateway.
   - *Payload*: `{ token }` (Google JWT Token string).
   - *Communication Path*:
     - Axios posts the token to `/auth/login`.
     - The Auth Service verifies the JWT with Firebase Admin, checks MongoDB, and generates a session UUID.
     - The Auth Service responds with the user document and sets a secure cookie named `session` holding the UUID.
   - *State Update*: `home.jsx` receives status 200 and calls `dispatch(setUserdata(data))` to write the user profile to Redux.

B. Session Restoration (Auto-Login)

1. **Session Hydration Check**:
   - *Frontend Function*: `getuser()` in `App.jsx` calling `getcurrentuser()` in `getcurrentuser.js`.
   - *Backend Service*: Gateway Service (Port 8000).
   - *Payload*: None. The browser automatically carries the cookie `session` in request headers.
   - *Communication Path*:
     - Axios makes a `GET` request to `/me`.
     - Gateway middleware `protect` intercepts the request, reads the `session` cookie, verifies it in Redis, and populates user context.
     - Gateway's `getCurrentuser` controller responds with the parsed user JSON.
   - *State Update*: `App.jsx` receives the profile payload and dispatches `setUserdata(data)` to update the global Redux store.

C. Conversation History Sync

1. **Listing Conversations**:
   - *Frontend Function*: `getconv()` in `sidebar.jsx` calling `getConversations()` in `getconversation.js`.
   - *Backend Service*: CHAT Service (Port 8002) proxied through the Gateway.
   - *Payload*: None. The cookie is carried in headers.
   - *Communication Path*:
     - Axios sends `GET` to `/chat/get-conversation`.
     - Gateway intercepts the cookie, checks Redis, appends `x-user-id` header with the database User ID, and proxies the request to the Chat Service.
     - Chat Service queries MongoDB and returns the list of conversations.
   - *State Update*: Sidebar receives the array and calls `dispatch(setConversations(data))` to render the list.
2. **Creating a Conversation**:
   - *Frontend Function*: `handleCreateConversation()` in `sidebar.jsx` calling `createConversation()` in `createconversation.js`.
   - *Backend Service*: CHAT Service (Port 8002) proxied through the Gateway.
   - *Communication Path*:
     - Axios calls `GET` to `/chat/create-conversation`.

    - Gateway performs cookie/Redis checks and forwards requests with the `x-user-id` header.
    - Chat Service generates a Mongoose conversation document and returns it.
  - *State Update*: Sidebar receives the conversation object and calls `dispatch(addConversation(data))` to prepend the chat bubble to the sidebar list.

D. Session Destruction (Logout)

1. **Terminating Sessions**:
   - *Frontend Function*: `handleLogout()` in `sidebar.jsx` calling `logout()` in `logout.js`.
   - *Backend Service*: Auth Service (Port 8001) proxied through the Gateway.
   - *Communication Path*:
     - Axios sends a request to `/auth/logout`.
     - Auth Service deletes the session key from Redis and clears the cookie `session` using `res.clearCookie("session")`.
   - *State Update*: Sidebar calls `dispatch(setUserdata(null))` to reset user profile data, closing the dashboard and triggering the login screen modal.

---

9. File-to-File Frontend Execution Workflow

Below is the step-by-step path detailing how frontend files interact during user sign-in:

```
[User Action] Click "Continue With Google" in home.jsx
   │
   ├──> Calls googlelogin() function.
   │    Uses config in utils/firebase.js to trigger Google Sign-in popup.
   │    Obtains user credentials and extracts JWT token: user.getIdToken().
   │
   ├──> Passes token to handlelogin(token) in home.jsx.
   ▼
[API Request] home.jsx → utils/axios.js
   │  ──> axios.js posts payload to "/auth/login" on Gateway Port 8000.
   │  ──> Backend processes login, writes browser cookie, and responds with user profile da
   ▼
[State Handler] home.jsx → src/redux/userslice.js
   │  ──> Receives response profile payload.
   │  ──> Triggers Redux dispatch: dispatch(setUserdata(data)).
   ▼
[UI Updates] home.jsx & src/components/sidebar.jsx
      ──> useSelector((state) => state.user.userData) detects profile update.
      ──> home.jsx unmounts the login overlay.
      ──> sidebar.jsx mounts and renders user profile card and email details.
```